

Take-Home Challenge — Senior Full Stack Developer

Role: Senior Full Stack Developer, R&D.

Estimated effort: 5-6 focused hours. Spread over up to **3 calendar days**.

Stack expected:

- **Extension:** Manifest V3 (TypeScript, strict), React (functional components + hooks), Zustand, Zod.
- **Backend:** Node.js (Express / Fastify / Nest — your call), TypeScript strict, a real persistent store (SQLite / Postgres / MongoDB — your call), JWT-based email/password auth, unit tests on handlers/services.

Submission: Public GitHub repo link + a 10 min video walking through your design, trade-offs, and a live demo.

Why this challenge

Apty ships a Digital Adoption Platform whose primary surface is a **browser extension** that overlays in-app guidance (walkthroughs, tooltips, validations, announcements) on top of third-party SaaS apps the customer's end-users are already using. The hard parts of this product are not the UI — they are: targeting elements reliably on apps we don't own, surviving SPA route changes and re-renders, keeping a service worker alive long enough to do useful work under MV3, and shipping a UX that doesn't fight the host page.

This challenge is a deliberately small slice of that real problem — an extension plus a backend, nothing else. We want to see how you **reason about** the runtime, the DOM, MV3, and the seams between the client and a real API. We care more about depth in a few areas than breadth across many; the rubric calls out where to spend your time.

The Challenge — "DAP Extension"

Build a **Chrome Manifest V3 extension** called **Mini Apty** plus a **Node.js backend**. Three parts.

Part A — Author mode

Let an "author" capture a sequence of **steps** on any public website. A step is:

- A pointer to a DOM element on the page (button, input, link, etc.)
- A short title and description shown to the end-user

- An optional "advance trigger" — `next-button` , `click-target` , or `input-change`

The author UI may live in a **popup, side panel, or injected drawer** — your call. Authors should be able to:

1. Start recording on the current tab.
2. click elements to capture them (with visible affordance).
3. Edit title/description for each captured step.
4. Save the walkthrough **to the backend and extension storage**, keyed by the page's origin + path pattern. Local cache in extension storage is fine, but the backend is the source of truth.

Part B — Preview mode

When an "author" click on saved walkthrough from the list, show step by step guide to on the screen based on the selected steps

1. Loads the walkthrough **from the backend** (with sensible caching so the player works offline once loaded).
2. Renders a **step balloon** anchored to the current step's element, with `Next` / `Previous` controls.
3. Advances on the configured trigger.
4. Persists progress so refreshing or navigating away and returning resumes from the correct step.

Part C — Node.js backend

Build a Node.js service that the extension talks to. A simple login form in the extension's popup/side-panel is enough — no separate web app needed. The backend must:

1. Expose a **JSON REST API** for walkthroughs (minimum: `POST /walkthroughs` , `GET /walkthroughs?origin=...&path=...` , `GET /walkthroughs/:id` , `PUT /walkthroughs/:id` , `DELETE /walkthroughs/:id`) and **auth endpoints** (`POST /auth/signup` , `POST /auth/login`).
2. **Persist** walkthroughs and users to a real store (SQLite / Postgres / MongoDB — your call). In-memory `Map` does not count. Manage the DB **connection / pool lifecycle** properly (graceful shutdown, no leaks on hot-reload).
3. **Authentication**. Email/password signup + login returning a **JWT**.
4. **Authorization**. Walkthroughs are scoped to the authenticated user; they can only be read/edited/deleted by their owner. A 401 vs 403 distinction enforced consistently.
5. Be **runnable locally** via `docker compose up` .

Hard requirements

- **Manifest V3.** Service worker, not background page.
- **Robust element targeting.** A plain `document.querySelector('div > div:nth-child(3)')` will not survive a re-render. You must design and justify a strategy that's better — and explain the trade-offs in your README.
- **Cross-origin storage via backend.** Walkthroughs are persisted on the Node.js backend and keyed by origin + path pattern. A walkthrough created on `app.example.com` must be retrievable when the user returns to `app.example.com` — even after browser restart, from a different Chrome profile, or from a different machine pointing at the same backend.
- **Network failure tolerance.** The player must not hard-crash if the backend is unreachable mid-session — degrade gracefully (cached load, queued writes, or a clear error state — your call, justify in README).
- **No clashes with the host page.** Your overlay must not be broken by, and must not break, the host page's styles, z-index, or event handlers.
- **TypeScript.** Strict mode on in both packages.
- **Error handling — extension.** A React error boundary on the overlay, an API-layer error normalizer, and user-visible messaging that distinguishes network / auth / validation / unknown failures.
- **Build that runs.** A reviewer must be able to clone, `pnpm i`, then:
 - `pnpm --filter backend dev` to start the API (with `docker compose up` first if your store needs it).
 - `pnpm --filter extension build` to produce the `dist/` folder to load as an unpacked extension in Chrome. A monorepo (`pnpm workspaces`) is required. Provide a `.env.example` covering JWT secret and DB URL.

What we'll be looking at

You will be evaluated against the rubric below. We care **a lot** more about depth in a few areas than breadth across all of them.

1. Architecture & MV3 understanding (25%)
 2. Element targeting & SPA / mutation handling (25%)
 3. Backend — auth, authZ, DB, error handling (25%)
 4. React / overlay quality & error handling (10%)
 5. Code quality & engineering hygiene (5%)
 6. README & trade-off articulation (10%)
 7. Bonus (up to +5%)
-

How to submit

Reply to the recruiter email with:

1. **Public GitHub repo URL** (or zip if your employer forbids public repos — include a `.git` directory either way so we can see history).
2. **Video link** (~10 min). Cover: (a) live demo of signup/login → author + player flows on a real public website, hitting the live backend, (b) walk us through your targeting strategy and your auth + middleware stack, (c) what you'd do differently with more time.
3. **One paragraph** in the email on the single hardest decision you made and why.

We do not review submissions blind — we review them in context of your prior screening.

Optimize for depth, not feature count.

Ground rules

- **AI assistance is allowed and expected.** We use it ourselves. But you must be able to defend every architectural decision on a follow-up call — "Cursor wrote it" is not an answer we will accept for trade-off questions.
 - **No NDA'd or proprietary code.** Don't paste in something from your employer.
 - **Don't optimize for the rubric over the product.** If the rubric and good judgment disagree, follow good judgment and tell us in the README.
-

After submission

If the submission clears the bar, we'll schedule a **60-minute follow-up call** where 1–2 Apty engineers will:

- Walk through your code with you.

- Ask you to extend the extension live (small, scoped change).
- Probe the trade-offs you made.

This is the final stage before an offer conversation. Good luck.